Лабораторная работа № 1

# Подготовка среды разработки и репозитория защищённого программного проекта

| | |
|---|---|
| Дисциплина: | Основы разработки защищённого программного обеспечения и компьютерных сетей |
| Вариант: | 19 — Система учёта транспортных средств организации |
| Версия сквозного проекта: | v0.1 |
| Студент: | Мостовая Полина Андреевна |
| Группа: | ИУСбд-02-23 |

2026

# Содержание


1 Тема и цель работы 2

2 Индивидуальный вариант 2

3 Описание выбранной программной системы 2

4 Перечень использованного программного обеспечения 3

5 Структура каталогов проекта 3

6 Назначение созданных файлов 4

7 Листинги конфигурационного модуля и точки запуска 4

7.1 `app/config.py` . . . . . 4

7.2 `app/main.py` . . . . . 5

8 Содержимое **`.gitignore`** и **`.env.example`** 5

8.1 `.gitignore` . . . . . 5

8.2 `.env.example` . . . . . 6

9 Результаты запуска приложения 6

10 Результаты выполнения тестов 7

11 Результаты проверки **`git status`** 7

12 История коммитов и метка версии v0.1 8

13 Проверки ошибочных настроек (самоконтроль) 8

14 Анализ реализованных мер безопасности 8

15 Ответы на контрольные вопросы 9

16 Вывод 10

17 Список использованных источников 11

# 1. Тема и цель работы

Тема: Подготовка среды разработки и репозитория защищённого программного проекта.

Цель работы: освоить первоначальную настройку среды безопасной разработки, создать структуру программного проекта, настроить виртуальное окружение и Git-репозиторий, а также реализовать минимальную запускаемую версию приложения без размещения конфиденциальных данных в исходном коде.

# 2. Индивидуальный вариант

Номер индивидуального варианта: 19.

Название разрабатываемой программной системы: «Система учёта транспортных средств организации».

Основные роли пользователей системы:

- водитель — просматривает закреплённые транспортные средства и передаёт сведения о поездках;
- диспетчер — ведёт учёт транспортных средств, назначает их водителям и контролирует их состояние;
- администратор — управляет учётными записями, ролями, настройками системы и событиями безопасности.

# 3. Описание выбранной программной системы

Разрабатываемая система предназначена для учёта транспортных средств организации. Система будет вести карточки транспортных средств, хранить сведения об их техническом состоянии и закреплении за водителями, а также поддерживать планирование и контроль поездок. К основным функциям относятся:

- регистрация, аутентификация пользователей и ролевое разграничение доступа;
- ведение реестра транспортных средств (государственный номер, модель, статус, технические данные);
- закрепление транспортных средств за водителями и регистрация поездок;
- поиск и фильтрация сведений о транспортных средствах и поездках;
- журналирование событий безопасности;
- формирование учётных отчётов.

Обрабатываемыми данными являются идентификационные данные и учётные записи пользователей, сведения о транспортных средствах, поездках и статусах, записи журнала аудита. К защищаемым активам относятся учётные записи пользователей, секретные ключи, прикладные данные, исходный код, журналы безопасности и база данных.

На данном этапе (v0.1) реализуется только первоначальная безопасная структура: загрузка конфигурации из переменных окружения, проверка секретного параметра и минимальная точка запуска приложения.

# 4. Перечень использованного программного обеспечения

| № | Программное обеспечение | Версия | Назначение |
|---|---|---|---|
| 1 | macOS | 26.6.2 | операционная система |
| 2 | Python | 3.12.14 | интерпретатор языка программирования в виртуальном окружении проекта |
| 3 | Git | 2.50.1 | система контроля версий |
| 4 | Visual Studio Code / терминал | — | среда разработки |
| 5 | python-dotenv | 1.2.2 | загрузка настроек из файла `.env` |
| 6 | pytest | 9.0.2 | средство автоматического тестирования |

Проверка установки выполнялась командами `python --version` и `git --version`. В системе использовался вызов `python` (Python 3.12.14).

# 5. Структура каталогов проекта

```
lab1_osn_po_seti/├──
 app/│
   ├── __init__.py│
   ├── config.py│
   └── main.py├──
 docs/│
   └── project_description.md├──
 tests/│
   └── test_main.py├──
 .env├──
 .env.example├──
 .gitignore├──
 README.md└──
 requirements.txt
```

Каталоги создавались в следующем порядке: сначала выполнялась инициализация Git-репозитория (`git init`), затем создавалось виртуальное окружение и после этого формировалась структура каталогов `app`, `docs`, `tests` с пустыми файлами-заготовками.

# 6. Назначение созданных файлов

| Элемент | Назначение |
|---|---|
| `app/` | каталог с исходным кодом приложения |
| `app/__init__.py` | файл, делающий каталог `app` пакетом Python |
| `app/config.py` | загрузка и проверка настроек приложения |
| `app/main.py` | точка запуска приложения |
| `tests/` | каталог с автоматическими тестами |
| `tests/test_main.py` | тесты первоначальной версии приложения |
| `docs/` | каталог проектной документации |
| `docs/project_description.md` | описание проектируемой системы |
| `.env` | реальные локальные настройки; не передаётся в Git |
| `.env.example` | безопасный шаблон настроек без реальных секретов |
| `.gitignore` | правила исключения файлов из Git-репозитория |
| `requirements.txt` | зафиксированный перечень зависимостей проекта |
| `README.md` | основная документация проекта |

# 7. Листинги конфигурационного модуля и точки запуска

## 7.1. `app/config.py`

Файл реализует загрузку настроек из переменных окружения и их проверку; библиотека `python-dotenv` загружает пары ключ–значение из `.env` в окружение процесса [3].

```
Загрузка
""" и проверка конфигурации приложения."""

import os
from dataclasses import dataclass

from dotenv import load_dotenv


@dataclass(frozen=True)
class Settings:
    Проверенные""" настройки приложения."""

    app_name: str
    app_env: str
    secret_key: str


def load_settings() -> Settings:
    Загрузить""" настройки из переменных окружения и проверить их."""
    load_dotenv()

    app_name = os.getenv("APP_NAME", Система" учета транспортных средств").strip
   ()
    app_env = os.getenv("APP_ENV", "development").strip().lower()
```

```
    secret_key = os.getenv("APP_SECRET_KEY", "")
    allowed_environments = {"development", "testing", "production"}

    if not app_name:
        raise RuntimeErrorНазвание(" приложения не может быть пустым")
    if app_env not in allowed_environments:
        raise RuntimeErrorНедопустимое(" значение APP_ENV")
    if len(secret_key) < 32:
        raise RuntimeError("APP_SECRET_KEY должен содержать не менее 32
   символов")

    return Settings(
        app_name=app_name,
        app_env=app_env,
        secret_key=secret_key,
    )
```

## 7.2. **app/main.py**

Файл содержит точку запуска приложения.

```
Точка
""" запуска системы учета транспортных средств."""

from app.config import load_settings


def build_startup_message(app_name: str, app_env: str) -> str:
    Сформировать""" сообщение без раскрытия конфиденциальных данных."""
    return f"{app_name}: приложение запущено в режиме {app_env}"


def main() -> None:
    Запустить""" приложение."""
    settings = load_settings()
    print(build_startup_message(settings.app_name, settings.app_env))


if __name__ == "__main__":
    main()
```

Основные особенности реализации:

- секретный ключ загружается из переменной окружения и не имеет опасного значения по умолчанию;
- минимальная длина секрета (не менее 32 символов) проверяется при запуске;
- допустимые режимы работы (`development`, `testing`, `production`) заданы явно;
- объект настроек объявлен как `frozen`-датакласс, то есть его значения нельзя изменить после создания;
- программа не выводит секретный ключ на экран и не записывает его в журнал.

# 8. Содержимое **.gitignore** и **.env.example**

## 8.1. **.gitignore**

```
# Виртуальное окружение
.venv/
venv/

# Конфиденциальная конфигурация
.env
.env.*
!.env.example

# Файлы Python
__pycache__/
*.py[cod]

# Результаты тестирования
.pytest_cache/
.coverage
htmlcov/

# Журналы
*.log
logs/

# Системные файлы
.DS_Store
Thumbs.db
```

Данный файл исключает из репозитория виртуальное окружение, файл реальной конфигурации `.env`, служебные файлы Python, результаты тестирования, журналы и системные файлы. Правило `!.env.example` позволяет хранить безопасный шаблон конфигурации.

## 8.2. `.env.example`

```
APP_NAMEСистема= учета транспортных средств
APP_ENV=development
APP_SECRET_KEY=replace_with_random_value_at_least_32_characters
```

В шаблоне приведены названия переменных и безопасное демонстрационное значение для `APP_SECRET_KEY`. На его основе был создан рабочий файл `.env`, в который записано сгенерированное командой `python -c "import secrets; print(secrets.token_urlsafe(32))"` случайное значение. Реальное значение ключа в отчёт не включается и в репозиторий не публикуется.

# 9. Результаты запуска приложения

Приложение запускалось командой:

```
python -m app.main
```

Полученный результат:

```
Система
 учета транспортных средств: приложение запущено в режиме development
```

Приложение успешно стартовало: конфигурация была загружена из файла `.env`, секретный ключ корректно проверен (длина не менее 32 символов), стартовое сообщение сформировано без вывода секретных данных.

## 10. Результаты выполнения тестов

Тесты запускались командой (pytest обнаруживает тесты по принятой структуре имён файлов и функций [4]):

```
python -m pytest -v
```

Результат выполнения:

```
tests/test_main.py::test_startup_message_contains_application_name PASSED [ 33%]
tests/test_main.py::test_startup_message_contains_environment PASSED     [ 66%]
tests/test_main.py::test_startup_message_does_not_contain_secret PASSED  [100%]

============================== 3 passed in 0.00s ===============================
```

Все три теста успешно пройдены (`3 passed`). Тесты проверяют, что стартовое сообщение содержит название приложения и режим работы, а также не содержит секретных значений.

## 11. Результаты проверки `git status`

После добавления разрешённых файлов в индекс выполнялась команда `git status`. В выводе отображались только подготовленные к фиксации файлы, файл `.env` и каталог `.venv` в списке отсутствовали. Итоговое состояние репозитория:

```
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

Дополнительно действие правил `.gitignore` подтверждено командами:

- `git check-ignore -v .env` → правило `.gitignore:6:.env`;
- `git check-ignore -v .venv` → правило `.gitignore:2:.venv/`;
- `git ls-files .env` — команда не вывела результата, что подтверждает, что файл `.env` не отслеживается Git.

В репозиторий вошли только разрешённые файлы:

```
.env.example
.gitignore
README.md
app/__init__.py
app/config.py
app/main.py
docs/project_description.md
requirements.txt
tests/test_main.py
```

# 12. История коммитов и метка версии v0.1

История репозитория (`git log --oneline`):

```
779fc15 Создана начальная безопасная структура проекта
```

Список меток (`git tag`):

```
v0.1
```

Работа выполнялась по стандартной последовательности: изменение файлов → добавление в индекс (`git add .`) → создание коммита → добавление метки версии (`git tag -a v0.1 -m "Начальная версия проекта"`). Коммиту соответствует начальная версия сквозного проекта v0.1.

# 13. Проверки ошибочных настроек (самоконтроль)

Для проверки корректности работы модуля конфигурации выполнены дополнительные проверки.

Проверка 1. Отсутствующий секрет. После временного удаления значения `APP_SECRET_KEY` из файла `.env` приложение завершилось с контролируемой ошибкой `APP_SECRET_KEY должен содержать не менее 32 символов`. Это подтверждает, что проверка длины секрета работает.

Проверка 2. Недопустимый режим работы. При установке `APP_ENV=unknown` конфигурация была отклонена приложением с сообщением `Недопустимое значение APP_ENV`.

Проверка 3. Защита от публикации секрета. Команда `git ls-files .env` не вывела результата, то есть файл `.env` не отслеживается Git.

После проверок корректные параметры конфигурации были восстановлены.

# 14. Анализ реализованных мер безопасности

- Секреты не хранятся в исходном коде. Секретный ключ загружается из переменной окружения через файл `.env`, а не прописывается в коде.
- Файл ‘.env‘ исключён из Git. Правила `.gitignore` и команда `git ls-files` подтверждают, что реальная конфигурация не попадает в репозиторий.
- Проверка конфигурации при запуске. Приложение проверяет название, режим работы и минимальную длину секретного ключа (не менее 32 символов).
- Отсутствие опасных значений по умолчанию. Если секрет не задан, используется пустая строка, и приложение корректно завершается.
- Неизменяемость настроек. Объект `Settings` объявлен как `frozen`-датакласс, что исключает случайное изменение параметров после загрузки.
- Секреты не выводятся пользователю. Стартовое сообщение содержит только название приложения и режим работы.
- Фиксация зависимостей. Перечень библиотек зафиксирован в `requirements.txt`, что обеспечивает воспроизводимость окружения.
- Изоляция окружения. Виртуальное окружение `.venv` не входит в репозиторий и может быть восстановлено из `requirements.txt`.

Публикация секретного ключа в открытый репозиторий могла бы привести к его компрометации и использованию злоумышленниками для подделки данных или доступа к системам, поэтому вопросу исключения конфиденциальных данных из репозитория в работе уделялось особое внимание.

# 15. Ответы на контрольные вопросы

1. Что включает в себя среда разработки программного обеспечения?

Среда разработки включает интерпретатор языка программирования, редактор исходного кода (IDE), систему контроля версий, менеджер зависимостей, средства тестирования и статического анализа, конфигурационные файлы и документацию проекта.

1. Для чего применяется система контроля версий Git?

Git позволяет сохранять историю изменений, сравнивать версии файлов, фиксировать законченные этапы разработки, возвращаться к предыдущим версиям, организовывать совместную работу, а также определять автора и время каждого изменения [1].

1. Чем рабочая директория отличается от индекса и репозитория?

Рабочая директория — это текущие файлы проекта, с которыми работает разработчик. Индекс — область, в которую файлы помещаются перед фиксацией (подготовленные к коммиту файлы). Репозиторий — хранилище истории сохранённых изменений (коммитов).

1. Для чего создаётся виртуальное окружение Python?

Виртуальное окружение изолирует зависимости проекта от других проектов и системных пакетов, что предотвращает конфликты версий библиотек, обеспечивает воспроизводимость окружения и упрощает развёртывание [2].

1. Почему каталог ‘.venv‘ не следует добавлять в репозиторий?

Каталог виртуального окружения содержит большое количество служебных файлов, не относящихся к коду проекта, и может быть восстановлен из списка зависимостей `requirements.txt`.

1. Какие сведения относятся к секретам программного проекта?

К секретам относятся пароли, токены доступа, ключи API, криптографические ключи, строки подключения к базам данных, учётные данные администраторов и закрытые сертификаты.

1. Почему пароли и ключи нельзя хранить непосредственно в исходном коде?

Исходный код обычно попадает в Git-репозиторий и может быть опубликован или просмотрен другими людьми. Это приводит к утечке конфиденциальных данных и компрометации системы.

1. Чем ‘.env‘ отличается от ‘.env.example‘?

Файл `.env` содержит реальные значения настроек, включая секреты, и хранится только локально. Файл `.env.example` содержит названия переменных и безопасные примеры заполнения и включается в репозиторий.

1. Для чего используется файл ‘.gitignore‘?

`.gitignore` задаёт перечень файлов и каталогов, исключаемых из репозитория: виртуальные окружения, конфиденциальную конфигурацию, служебные и системные файлы.

1. Почему секретные параметры необходимо проверять при запуске?

Проверка позволяет своевременно обнаружить отсутствующие или некорректные настройки и не допустить запуск приложения с небезопасной конфигурацией, например

с коротким или неустановленным секретным ключом.

1. Для чего фиксируются версии программных зависимостей?

Фиксация версий в `requirements.txt` обеспечивает воспроизводимость окружения: на любой машине будут установлены одинаковые версии библиотек, что исключает ошибки, связанные с несовместимостью версий.

1. Что представляет собой Git-коммит?

Коммит — это зафиксированное состояние проекта в определённый момент времени. Он сохраняет снимок изменений, автора, дату и сообщение о внесённых изменениях.

1. Для чего используется метка v0.1?

Метка (тег) `v0.1` отмечает в истории Git конкретную версию проекта — начальную безопасную структуру. В дальнейшем по метке можно будет вернуться к данной версии.

1. Какие файлы первоначальной версии проекта могут быть опубликованы?

Опубликованы могут быть файлы исходного кода, документация, шаблон конфигурации `.env.example`, список зависимостей и правила `.gitignore`. Не подлежат публикации файл `.env` и виртуальное окружение.

1. Какие последствия может вызвать случайная публикация секретного ключа?

Публикация ключа может позволить злоумышленникам подделывать данные, получать несанкционированный доступ к системам и сервисам, что приведёт к нарушению конфиденциальности, целостности и доступности информации.

# 16. Вывод

В ходе лабораторной работы была создана начальная версия защищённого программного проекта v0.1 «Система учёта транспортных средств организации» (вариант 19). Выполнены следующие задачи:

- настроено виртуальное окружение Python и установлены зависимости (`python-dotenv`, `pytest`);
- сформирована логически организованная структура проекта (`app`, `tests`, `docs`);
- реализовано минимальное запускаемое приложение с загрузкой настроек из переменных окружения и проверкой обязательного секретного параметра;
- созданы правила `.gitignore`, исключающие конфиденциальные и служебные файлы из репозитория;
- оформлены `README.md` и `docs/project_description.md`;
- создан автоматический тест, подтвердивший корректность работы приложения (`3 passed`);
- выполнен первоначальный коммит и зафиксирована метка версии v0.1.

Проверки подтвердили, что файл `.env` с реальным секретным ключом не отслеживается Git, а приложение корректно отклоняет некорректную конфигурацию. Освоены основные команды Git и базовые приёмы безопасной разработки. Подготовленная структура будет использоваться в последующих лабораторных работах сквозного проекта.

# 17. Список использованных источников


1. Git Project. Git Reference [Электронный ресурс]. URL: <https://git-scm.com/docs> (дата обращения: 15.10.2026).
2. Python Software Foundation. `venv` — Creation of virtual environments. Python 3.12 Documentation [Электронный ресурс]. URL: <https://docs.python.org/3.12/library/venv.html> (дата обращения: 15.10.2026).
3. Saurabh Kumar, Bertrand Bonnefoy-Claudet. python-dotenv [Электронный ресурс]. PyPI. URL: <https://pypi.org/project/python-dotenv/> (дата обращения: 15.10.2026).
4. pytest Development Team. pytest Documentation [Электронный ресурс]. URL: <https://docs.pytest.org/en/stable/> (дата обращения: 15.10.2026).